Foreign Whispers Report

Shreyon Roy
sr5655

I used Google Colab for this Foreign Whispers project since I have a MacOS computer and would not be able to run Nvidia. I used Python subprocesses and had to work around many of the dependency conflicts.

In phase 1, I went through the end to end pipeline to understand how the process works. Then, I went through phase 2 and the various integration notebooks to try to understand the pipeline better. I used an input video that I downloaded from youtube and stored it in my run time in Google Colab. I updated the video_registry.yml because I had some issues trying to download the video from the youtube URL. I ran a server with the command: !MPLBACKEND=Agg uv run uvicorn api.src.main:app --host 127.0.0.1 --port 8090 > api.log 2>&1 &. Then I used FWClient to create a transcript from the video. Then, for translations, the reranking task involved modifying reranking.py to get shorter translations. Generally, for every English translation, there is a longer Spanish translation so shorter equivalent phrases were used to replace longer phrases. Then, I tried splitting on clause separators like dashes, semicolons, and colons and keeping only the main clause. Then, I tried keeping only the text before the first comma. If it was still too long, the words were trimmed until they fit the character budget of 15 characters per second. Candidates are deduplicated and sorted by whether they fit the budget, then by length.

Diarization was tricky because many times when I tried to do this, it kept resulting in only one speaker label for all segments of the text. I used pyannote for this so I used pip install piannote.audio. I then sent a POST request to the server to diarize the input video. I had to use my hugging face token for this and I had to accept and agree to various different terms for Hugging Face account. I had to accept terms for different pyannote libraries like speaker-diarization-3.1, speaker-diarization-community, etc. I had to change use_auth_token to token in the diarization.py file. Within the file, assign_speakers function merges diarization output with Whisper transcription segments. For each transcription segment it computes temporal overlap with every diarization turn using max(0, min(seg_end, turn_end) - max(seg_start, turn_start)) and assigns whichever speaker had the most overlap. If there is no

overlap or diarization is empty, it defaults to SPEAKER_00. In the input video I used, I saw that this function worked correctly when it assigned 3 speakers the labels of SPEAKER_00, SPEAKER_01, SPEAKER_02. When I implemented this function from the integration notebook, I saw that it passed all of the 4 tests. I then used the diarize_audio() function and printed out the diarization segments. I saw that it had given the proper speaker label to each segment in the video. The diarize.py file checks for a cached result first, then if its not found, it extracts mono 16kHz audio from the MP4 using ffmpeg with the asyncio.to_thread to avoid blocking the event loop. Then it runs pyannote diarization and merges speaker labels into the Whisper transcription JSON on disk. I had difficulty with the frontend pipeline integration because I was working with Google Colab so I did not understand how I would implement it. I originally decided, I would try to make sure the full pipeline works properly first with an actual dubbed output video and then I would work on the front end. But then I saw that there were aspects of the pipeline that were difficult later on which proved that implementing the front end would be challenging. Then the next stage was alignment. For this, I mainly worked with the alignment.py file. It works in three layers. First, _count_syllables() strips accents from text and counts vowel clusters to estimate syllable count because each vowel cluster roughly equals one spoken syllable. Then _estimate_duration() divides syllable count by 4.5 (syllables/second for Spanish) to predict how long the TTS audio will be.SegmentMetrics is a dataclass that pairs one English segment with its Spanish translation and computes three key numbers: predicted_tts_s , predicted_stretch, and overflow_s. The decide_action() function maps the stretch ratio to ACCEPT, MILD_STRETCH, GAP_SHIFT, REQUEST_SHORTER, or FAIL using fixed thresholds. global_align() is the greedy scheduler. It goes through segments left to right, applies the policy decision for each one, and tracks cumulative drift like if segment 5 borrows 0.3s of silence, every segment after it shifts forward by 0.3s. Then, global_align_dp() is the improved version. Instead of always spending a silence gap the moment a segment qualifies, it looks 3 segments ahead and only uses the gap if the current overflow is at least 25% as large as the biggest upcoming overflow. This prevents wasting a large silence gap on a small overflow when a bigger one is coming.

The next step was implementing the Chatterbox TTS in Google Colab. When I originally did this, I kept getting errors because it kept falling back to Coqui TTS. I had to do a lot of research

to figure out how to use Chatterbox TTS in Google Colab. I then read that one has to downgrade the runtime to a 2025.07 version on Google Colab. Additionally, I had to use the following version pins: pip install "numpy==1.25.2" chatterbox-tts --no-deps pip install torch==2.6.0 torchaudio==2.6.0 transformers==4.46.3 \ librosa==0.11.0 conformer==0.3.2. I used the following import statements: from chatterbox.tts import ChatterboxTTS and from chatterbox.mtl_tts import ChatterboxMultilingualTTS. During the process, there were a lot of difficulties with the Chatterbox server because it kept going down and trying to debug it was very time consuming. I had to read the api logs frequently to try to understand what was happening in the process. After lots of tries and facing lots of errors, I sent a post request to http://localhost:8088/api/tts/input_video which created a WAV file. Eventually, I sent a post request to stitch the input video. The stitching process was not difficult. The video was then stored in a pipeline data folder under a dubbed videos sub directory as an MP4 file. The tts.py used a FastAPI router for POST /api/tts/{video_id}. It accepts config id, alignment boolean value, and a speaker_wav. It loads the translated JSON, extracts unique speaker IDs from the segments, builds a speaker voice path mapping using resolve_speaker_wav, then delegates to TTSService.text_file_to_speech(). Then it returns the WAV path and the speaker voice mapping. There is a function resolve_speaker_wav() in voice_resolution.py that implements checks for speakers/{lang}/{speaker_id}.wav first, then speakers/{lang}/default.wav, then speakers/default.wav. Then it returns a relative path string because Chatterbox expects paths relative to its /app/voices/ mount point. Raises FileNotFoundError if nothing is found at all.

Overall, once the output dubbed video was stitched, I saw that the dubbing had worked, where all of the speakers had different voices and the voices were successfully converted from English to Spanish. However, I noticed one problem which was that all the voices had a male voice. Even though the voices were different, the voices could not distinguish between a male and female voice.

When reflecting on the project, I realized a lot of issues with the approach that I took. The project was designed for a Docker environment with Nvidia. There were many dependency version conflicts (numpy, scipy, torchaudio, etc.), no persistent storage (requiring the Drive backup loop), and no Docker. The actual assignment tasks were understandable but for me, most of the efforts came from trying to debug the infrastructure. The interesting thing that I learned

during the project is that stages such as downloading, transcribing, diarization, etc. required one set of dependencies but the Chatterbox TTS required a different set of dependencies so this caused a lot of conflicts. I had to restart my whole runtime after I did the diarization and alignment portions just to enable Chatterbox TTS because the dependencies were not aligned. I think if I had used a public Docker image, that could have been more optimal.
Additionally, since I was using Google Colab, the runtime would sometimes end which would cause me to have to rerun the commands again and reset many variables. Sometimes this would cause dependency issues too. Additionally, since I chose to edit many of the .py files in Google Colab's editor, it caused many modified files to disappear if the runtime ended. I had to keep using copy commands to update the files that I cloned from the repository. As a result, going through the whole pipeline was challenging and time consuming. I believe the ideal way to do this project would have been to use a Cloud VM since I use MacOS.
Both the input and output video are saved to the Github repository as mp4 files.